Exercise 6: Function Approximation

Teoman Kaman

1. Monte Carlo vs TD (Mountain Car)

Monte Carlo methods perform worse than Temporal Difference methods in the Mountain Car task.

The reason is that Monte Carlo updates only at the end of an episode, while TD methods update during the episode using bootstrapping. In Mountain Car, rewards are delayed, so learning from intermediate states is very important.

TD methods can update values using the next state's estimate, which reduces variance and improves learning speed. Monte Carlo must wait until the episode ends, so its updates are slower and more noisy.

Therefore, TD methods usually perform better than Monte Carlo methods in this task.

2. Semi-gradient Expected SARSA

(a) Pseudocode

```
Initialize w
For each episode:
    Initialize state S
    While S not terminal:
        Choose A using epsilon-greedy from q_hat(S,a,w)
        Take action A, observe R, S'
        If S' terminal:
            U = R
        Else:
            U = R + gamma * sum_a pi(a|S') q_hat(S',a,w)
            where pi is the epsilon-greedy policy w.r.t. q_hat(s,a,w)
        w = w + alpha * (U - q_hat(S,A,w)) * f(S,A)
            where f(S,A) = grad_w q_hat(S,A,w) for linear approximation
        S = S'
```

(b) Q-learning change

We replace the expected update target with the greedy maximum:

$$U = R + \gamma \max_a \hat{q}(S', a, w)$$

This makes Q-learning off-policy: the target uses the greedy action regardless of the behavior policy used to select actions. Everything else in the pseudocode stays the same.

3. Four Rooms

(a) Tabular aggregation

Each state-action pair is represented using one-hot encoding. This is equivalent to tabular SARSA.

The results match tabular learning, so the implementation is correct.

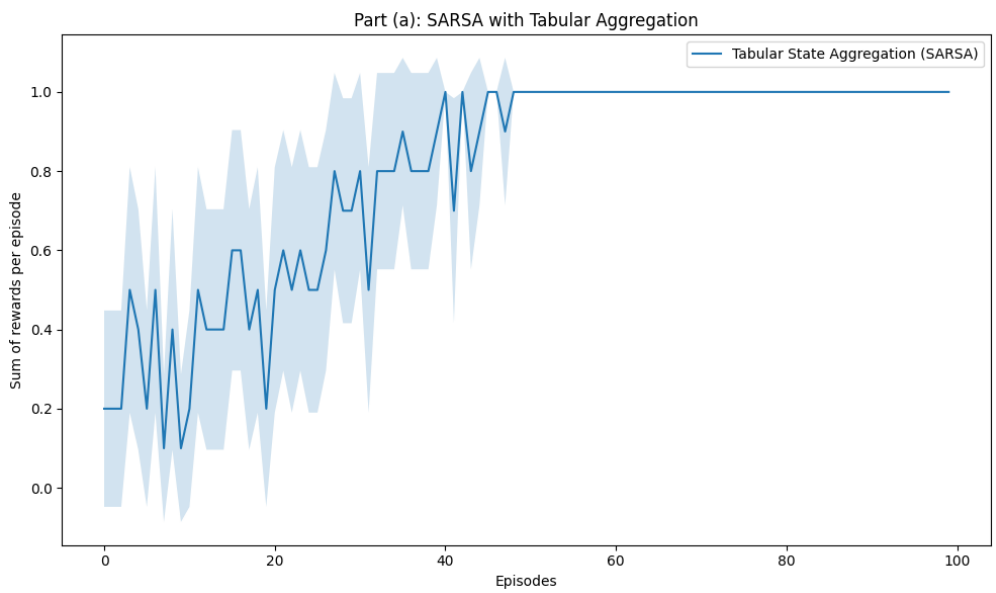

Figure 1: Tabular SARSA

—

(b) Aggregation methods

Tile-based aggregation improves learning speed because nearby states share information. Room-based aggregation is too coarse and loses important differences between states. So aggregation helps, but too much aggregation can hurt performance. Specifically, over-aggregation causes *aliasing*: states with fundamentally different optimal actions are forced to share the same feature representation, making the true Q-function unrepresentable within the function class. Therefore, we should not always aggregate states as much as possible.

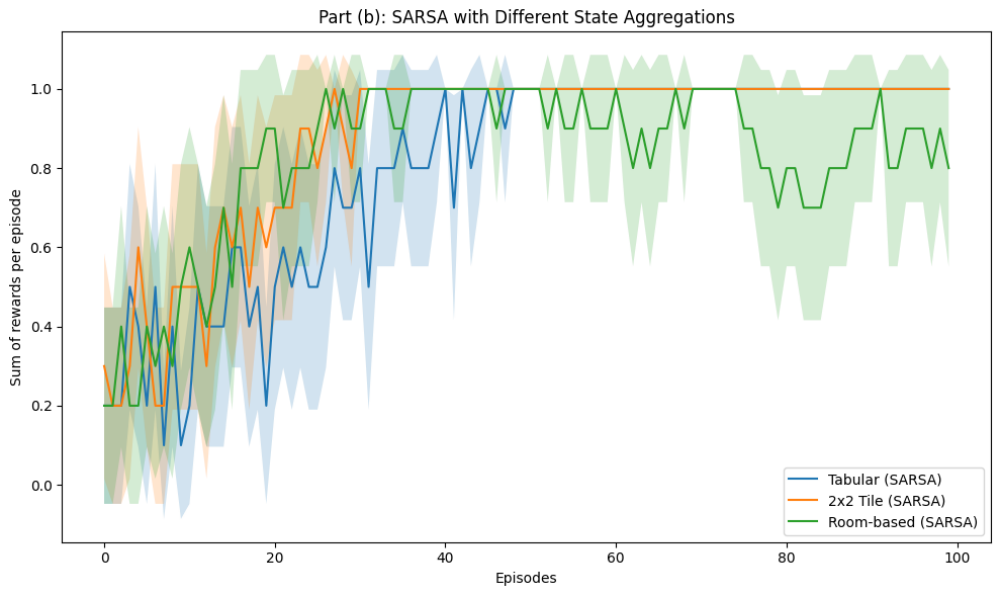

Figure 2: Tabular SARSA different aggregations

—

(c) Q-learning (extra)

Q-learning can learn faster because it uses the max operator.

However, it can be less stable than SARSA due to optimistic updates.

Overall, both methods work, but feature choice matters more.

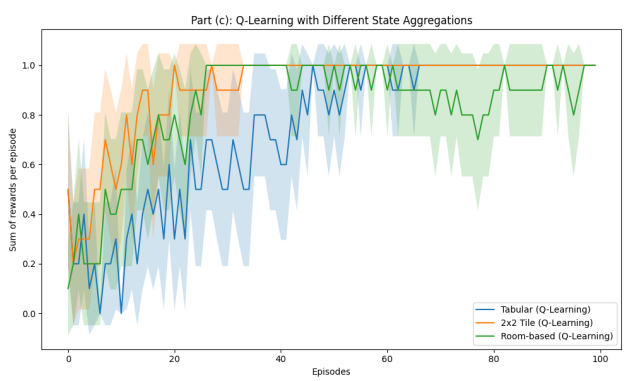

Figure 3: SARSA vs Q learning

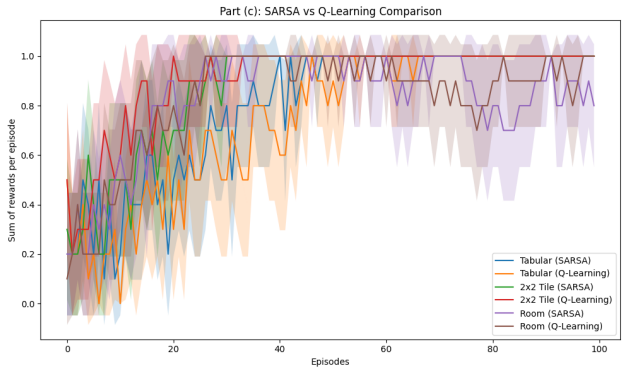

Figure 4: SARSA vs Q learning

—

The constant feature is necessary as a bias term. Without it, the approximation is constrained to $Q(s, a) = w_1 x + w_2 y$, which forces Q-values to be zero whenever both coordinates are zero and prevents the model from learning a non-zero baseline value independent of position. This severely limits expressiveness near the origin. Actions are handled using separate weight blocks per action. Linear features perform worse than aggregation because they cannot capture walls and structure.

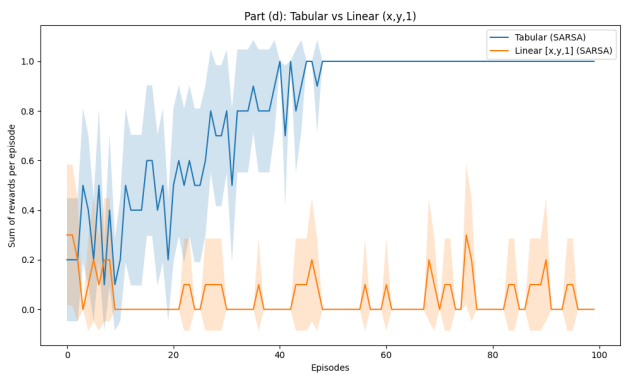

Figure 5: Tabular vs Linear

—

Normalized features give a small improvement because scaling coordinates to $[0, 1]$ keeps all feature magnitudes comparable, preventing large raw coordinate values from dominating gradient updates and destabilizing learning. Distance feature improves performance because the shortest-path distance to the goal encodes task-relevant structure directly: states closer to the goal should have higher Q-values, and this feature makes that relationship linearly representable. This is domain knowledge that raw coordinates alone cannot capture due to walls. Feature design strongly affects performance.

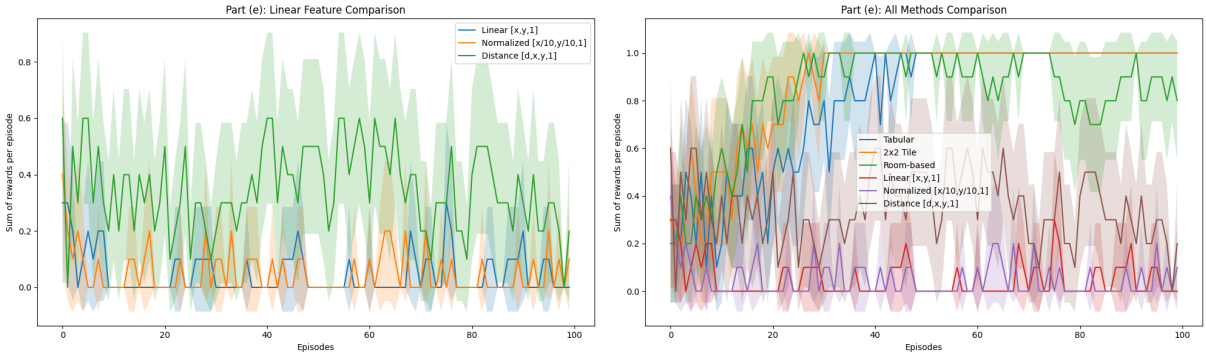

Figure 6: Feature comparison

—

(f) Larger environments (extra)

Function approximation helps in larger state spaces.

Tile-based features scale better than simple linear features.

Simple features are too weak for complex environments.

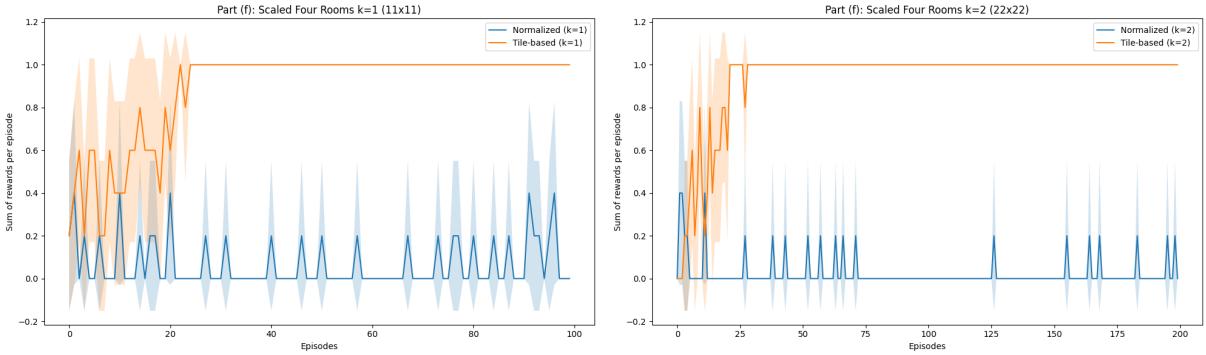

Figure 7: Scaled environment